

OPERATING SYSTEMS (UE24CS242B)

Unit 1 – Introduction & Computer System Organization

Question and Answers

Lecture 1 - Introduction

Q1. You are designing an Operating System for a supercomputer used exclusively for weather forecasting (a scientific mainframe) and another for a home entertainment tablet. Contrast the "Operating System Goals" for these two systems. How does the definition of "efficiency" differ between them?

Answer: Mainframe (Weather Supercomputer): The primary goal is **Resource Utilization**. Since the hardware is expensive and powerful, the OS must ensure the CPU, memory, and I/O channels are busy constantly to maximize throughput. "Efficiency" here means executing the maximum number of calculations per second.

Tablet (Home Entertainment): The primary goal is **User Convenience** and **Ease of Use**. The OS focuses on a responsive UI and battery life rather than keeping the CPU 100% busy. "Efficiency" here means low power consumption and smooth interaction, even if the CPU sits idle waiting for user input.

Q2. Why is the Operating System described as "Interrupt Driven"? What would happen to the system's responsiveness if interrupts were disabled entirely?

Answer: Interrupt Driven: The OS sits idle until something happens. Events (keystrokes, disk completion, network packets, system calls) are all signaled via interrupts. The OS wakes up, handles the event, and goes back to waiting or running a user process.

Consequence of Disabling: If interrupts were disabled, the OS could not react to any external input. The keyboard would freeze, the mouse wouldn't move, and disk operations wouldn't notify the CPU upon completion. The system would effectively appear "hung" or unresponsive to the user.

Q3. "Main memory can be viewed as a cache for secondary storage." Explain this statement. If the CPU needs data that is currently on the Hard Disk, describe the chain of copying required to get that data into the CPU Registers.

Answer: Caching involves copying data from slower storage to faster storage. RAM is faster than Disk, so it acts as a cache for the Disk.

Chain of Copying:

1. Data is located on the **Hard Disk** (Secondary Storage).

2. It is copied into **Main Memory (RAM)** via I/O transfer.
3. It is loaded into the **Cache (L1/L2/L3)**.
4. Finally, it is moved into the **CPU Registers** for the instruction to process it.

Q4. A CPU is modifying a variable X stored in the Cache. The same variable X exists in Main Memory. What is the "Cache Coherency" problem here? If an I/O device reads X directly from Main Memory before the CPU writes it back, what happens?

Answer: The CPU has an updated value of **X** (e.g., 50) in the Cache, but Main Memory still holds the old value (e.g., 10). This is an inconsistency.

If an I/O device (using DMA) reads X from Main Memory, it reads the **stale value (10)** instead of the correct value (50).

This leads to **data corruption or incorrect behavior** (e.g., saving the wrong balance to a bank account). The OS and hardware must ensure the cache is flushed or synchronized before such I/O occurs.

Q5. "A user program cannot directly access hardware." If a text editor needs to save a file to the disk, explain the mechanism it uses to bypass this restriction securely. What role does the Dual-Mode (User/Kernel) architecture play here?

Answer: The text editor issues a **System Call** (e.g., write()). This is a special instruction that triggers a software interrupt (Trap).

Dual-Mode Role:

- The trap switches the CPU from **User Mode** to **Kernel Mode**.
- The OS (now in Kernel Mode) validates the request (permissions, path).
- The OS performs the hardware operation on the disk.
- The OS switches back to **User Mode** and returns control to the text editor.
- This ensures the user program never touches the hardware directly, preventing errors or malicious access.

Q6. "Multiprocessor systems are becoming common." Briefly explain the difference between a Uniprocessor system and a Multiprocessor system. Why might a Multiprocessor system be preferred for a server handling thousands of transactions?

Answer: Uniprocessor: Has a single main CPU capable of executing one instruction stream.

Multiprocessor: Has two or more CPUs sharing the computer bus, clock, and memory.

Preference: For a server, a Multiprocessor system provides **Increased Throughput**. By executing multiple transactions in parallel on different CPUs, the system can handle a much higher load than a single processor, even if that single processor were very fast. (Note: Graceful degradation is another benefit).

Lecture 2 - OS Structure and Operations

Q1. In a Symmetric Cluster, two nodes try to write to the same student record on the shared Storage Area Network (SAN) at the exact same instant. What mechanism prevents data corruption here? Why isn't a simple local semaphore sufficient?

Answer: Since the nodes are separate computers, they do not share local memory. A local semaphore on Node A is invisible to Node B. Thus, the system must use a **Distributed Lock Manager (DLM)**.

Mechanism: The DLM coordinates access control across the network, ensuring that only one node holds the "write lock" for a specific shared resource on the SAN at any given time.

Q2. In a multiprogrammed system, multiple jobs reside in memory simultaneously. If the system has 8GB of RAM but the active jobs require 12GB total, what mechanism allows the system to continue operating? Briefly explain Swapping/Virtual Memory.

Answer: Constraint: Physical memory is finite. Not all jobs can fit in RAM at once.

Mechanism: The OS uses **Swapping** or **Virtual Memory**.

Operation: Processes not currently executing are swapped out to the hard disk (backing store), freeing up RAM for the active process. When the swapped-out process is needed again, it is loaded back into memory. This allows the execution of processes larger than physical memory.

Q3. Scenario: What would happen if the **Mode Bit** stuck at '0' due to a hardware glitch while a user program was running? Analyze the risk to system stability and security.

Answer: State: If the bit sticks at 0, the CPU remains in **Kernel Mode** permanently.

Risk: The currently running user program would have **unrestricted access** to the entire system.

Consequence: It could read other users' passwords from memory, wipe the hard disk, or overwrite the OS code. The system loses all protection boundaries, likely leading to a crash or security breach.

Q4. Modern CPUs often support more than just two modes. What is the purpose of the VMM (Virtual Machine Manager) Mode? How does it relate to "Guest VMs"?

Answer: In virtualization, a host OS runs multiple guest operating systems. CPUs support a dedicated mode for the **Virtual Machine Manager (Hypervisor)**.

This allows the VMM to have higher privileges than the Guest OS kernels (which think they are in kernel mode but are actually being managed) without needing full physical root access for every operation, improving virtualization efficiency and security.

Q5. A system administrator sets the Timer interrupt to occur every 1 microsecond (very fast). What is the negative impact of this configuration on the system's "Throughput"?

Answer: Throughput is the number of jobs completed per unit of time. Setting the timer too fast causes an **Interrupt Storm**. The CPU spends the vast majority of its time saving context, jumping to the OS, handling the timer, and restoring context (Overhead). Very little time remains to actually execute the user program instructions, causing the system's effective throughput to drop drastically.

Lecture 3 - Kernel Datastructures

Q1. The Linux Kernel scheduler needs to pick the next task to run from a pool of ready tasks. Why does the OS use a Balanced Binary Search Tree (like a Red-Black Tree) instead of a simple Linear List? Explain the impact on system performance using "Big O" notation.

Answer: In a linear list, finding a specific task or inserting a task based on priority takes $O(n)$ time. As the number of tasks (n) grows, the scheduler becomes slower. A Balanced Binary Search Tree guarantees a tree height of roughly $\log n$.

This ensures that searching for the highest priority task or inserting a new one takes only $O(\log n)$ time. This speed is critical for keeping the system responsive.

Q2. "A Distributed System creates the illusion of a single system." Explain this statement. If you are on a workstation in a generic distributed network, do you know which specific physical machine is storing your file?

Answer: Users see a unified interface (e.g., a file system) and do not need to know the physical location of resources. The system is a collection of separate, heterogeneous systems networked together.

No, the user generally does not know (or care) if the file is on their local disk or a server in another building; the **Network Operating System** handles the message exchange to retrieve it transparently.

Q3. A bank wants to move to the cloud but is worried about data privacy. Would you recommend a Public, Private, or Hybrid cloud? Justify your answer.

Answer: I would suggest either **Private Cloud** or **Hybrid Cloud**.

Justification:

- **Private Cloud:** The infrastructure is run solely for the company. This offers maximum security and control over sensitive financial data, unlike a Public Cloud which is available to anyone willing to pay.
- **Hybrid Cloud:** They could keep sensitive data on the private cloud and use the public cloud for non-sensitive, high-volume traffic.

Q4. Why is a Real-Time Operating System (RTOS) necessary for a car's Anti-lock Braking System (ABS)? Task: What happens if the OS decides to "pause" the braking process for 2 seconds to save a file to disk? Define "Fixed Time Constraints".

Answer: An RTOS has well-defined, **fixed time constraints**. Processing *must* be done within the constraint.

If the braking command is delayed (even by seconds), the operation is considered a **failure**, potentially causing an accident. Correct operation depends on meeting the deadline, not just the logical result.

Q5. Comparing Virtualization approaches, What is the difference between a VMM that runs natively versus one that runs on a Host OS? Which is "Type 1" and which is "Type 2"?

Answer:

- **Native (Type 1):** The VMM runs directly on the hardware. In this case, the VMM *is* the host operating system. This is typically more efficient for data centers.
- **Hosted (Type 2):** The VMM runs as an application inside a standard Host OS (like macOS running Windows in a window). This is better for developer exploration.

Lecture 4 - OS Services, Design and Implementation

Q1. A headless server (a computer without a monitor or keyboard) in a data center crashes due to a memory error. Since there is no screen to display an error message, which OS service is responsible for handling this, and how does the administrator eventually find out what happened?

Answer: The **Error Detection** service is responsible. When the error occurs, the OS generates a core dump or writes the error details to a system **log file**.

The administrator accesses these logs later (via network or remote shell) to diagnose the issue. Without this service, the system would simply freeze with no diagnostic trail.

Q2. A program needs to write a 10MB video file to the disk. Why can't the system pass this 10MB of data directly through the CPU registers during the system call? What method is used instead?

Answer: CPU registers are very small (typically 64 bits) and few in number. They cannot hold 10MB of data.

The **Block/Address method** is used. The program stores the 10MB data in a memory block and passes only the **starting memory address (pointer)** of that block in a register. The OS then reads the data from that memory location.

Q3. You have written a program main.c. Before it becomes a running process, it goes through several steps. Explain the roles of the Compiler, Linker, and Loader in this process.

Answer:

1. **Compiler:** Translates source code (main.c) into object code (machine language).
2. **Linker:** Combines the object code with required libraries (and other object files) into a single binary executable.
3. **Loader:** When the user runs the program, the loader loads the binary into Main Memory and starts execution.

Q4. Early OSes were written in Assembly. Modern OSes (like Linux/Windows) are written mostly in C/C++. Give two reasons for this shift. Is there any part of the OS still written in Assembly?

Answer: C code is **portable**. C code can be recompiled for different CPU architectures (x86, ARM). Assembly is specific to one CPU family.

It is also highly **maintainable**. High-level code is easier to read, write, and debug.

Yes, small critical sections like the **interrupt handlers** and **device drivers** are still written in Assembly for maximum speed and direct hardware control.

Q5. Two processes need to exchange data. The OS provides Shared Memory and Message Passing. Which method would you choose for exchanging small amounts of status data between two computers across a network? Justify.

Answer: Message Passing.

Justification: Shared Memory is difficult or impossible to implement across different physical computers (distributed systems). Message passing is designed for communication, handling the packet formatting and synchronization required for network data transfer.

Q1. How does the "Process Lifecycle" in a mobile OS (like Android) differ from a traditional desktop OS regarding background processes? Why might the OS terminate a background process without the user explicitly closing it?

Answer: Mobile devices have limited memory and battery compared to desktops. While desktop systems typically use swapping (virtual memory) to manage overflow, mobile OSes generally do not support swapping.

Instead, when memory runs low, the mobile OS automatically **terminates** background processes (that are not visible to the user) to reclaim memory for the foreground application. The state is saved to flash memory to be restored later.

Q2. When the CPU switches from Process P0 to Process P1, it performs a "State Save" and "State Restore." Where exactly is this state saved? List four specific items stored in this data structure.

Answer: The state is saved in the **Process Control Block (PCB)** of Process P0.

Items Stored:

1. **Program Counter:** The address of the next instruction to execute.
2. **CPU Registers:** Accumulators, index registers, stack pointers.
3. **CPU Scheduling Information:** Process priority, pointers to scheduling queues.
4. **Memory-Management Information:** Base/limit registers or page tables.

Q3. "Context Switching is considered pure overhead." Justify this statement. Why don't we switch processes every single microsecond to create "perfect" multitasking?

Answer: During a context switch (saving P0's state and loading P1's state), the system is doing administrative work, not executing any user instructions. No useful work is accomplished during this transition time.

If we switch too frequently (e.g., every microsecond), the CPU would spend 100% of its time saving/restoring registers and 0% of its time running the actual programs, bringing the system to a standstill.

Q4. A system is running 10 processes. 9 of them are "I/O Bound" and 1 is "CPU Bound." Describe the behavior of the Ready Queue. Is it likely to be full or empty most of the time?

Answer: **I/O Bound** processes spend most of their time waiting in Device Queues. They run on the CPU for a tiny burst and then leave while the **CPU Bound** process stays in the Ready Queue/CPU.

The Ready Queue will likely be **short or empty** most of the time. The 9 I/O processes are constantly waiting for devices. The single CPU-bound process will hog the CPU, while the I/O processes quickly cycle through the Ready Queue and back to Device Queues.

Q5. System Calls: Explain the relationship between fork() and exec(). Task: Why do most programs call exec() immediately after fork()? What happens to the memory of the child process during exec()?

Answer: fork() creates a duplicate copy of the parent process. exec() replaces the memory space of a process with a new program. Programs call them in sequence to run a different program. For example, the Shell forks a copy of itself, then the child calls exec() to become the "ls" command.

During exec(), the child's text, data, and stack segments are completely destroyed and overwritten by the new program loaded from the disk.

Q6. What is "Cascading Termination"? If a user kills the main BASH shell process, what happens to the background commands running under it in an OS that enforces cascading termination?

Answer: Cascading Termination is a policy where the OS does not allow a child process to exist if its parent has terminated.

If the BASH shell (Parent) is killed, the OS detects its children. The OS automatically initiates termination for all children, grandchildren, and further descendants of that shell. All background commands running under it are forcibly closed.

Q7. You run the command ps -el and see a process with status Z (Zombie). Explain exactly what this process is. Is it using CPU? Is it using Memory? Why is it still in the table?

Answer: A Zombie is a process that has completed execution (called exit()) but whose parent has not yet called wait().

It is **not** using CPU and has freed most of its memory.

It remains in the Process Table solely to store its **Exit Status** (return code). It stays there until the parent reads this status via wait(), allowing the OS to delete the entry.

Q8. You are using a Bounded Buffer (fixed size) for IPC. What must the Producer do if the buffer is full? What must the Consumer do if the buffer is empty?

Answer: If the buffer is full, the Producer must **Wait** (block) until there is free space (i.e., until the Consumer removes an item).

If the buffer is empty, the Consumer must **Wait** (block) until there is data available (i.e., until the Producer adds an item).

They must coordinate to prevent overwriting data or reading garbage.

Lecture 7 - System calls for Process Management & Life cycle of I_O request

Q1. You are optimizing a system where a process needs to create a child that *immediately* calls `exec()` to run a new program. Would you use `fork()` or `vfork()`? Justify your choice based on memory overhead and performance.

Answer: `vfork()` is preferred in this specific scenario.

Justification:

- **Standard `fork()`:** Copies the parent's data space, heap, and stack to the child. If the child immediately calls `exec()`, this copying is wasted effort because `exec()` replaces all of it anyway.
- **`vfork()` Optimization:** It creates a child without copying the address space. The child borrows the parent's memory until it calls `exec()`, making it significantly faster and more memory-efficient for this specific pattern.

Q2. A server runs for months. You check the process table and see hundreds of "Zombie" entries. Explain what a Zombie process is in terms of resource usage. Why is it a problem if they accumulate?

Answer: A Zombie is a process that has terminated (`exit()`) but whose parent has not yet called `wait()` to collect its status. They consume no CPU and almost no memory (text/data are freed).

They still occupy a slot in the **Process Table** (to store the PID and exit status). If zombies accumulate, the system runs out of available PIDs, preventing new processes from starting.

Q3. "During I/O execution, the requesting process is typically blocked." Task: Why does the OS block the process instead of letting it spin in a loop checking for data? How does this improve "System Throughput"?

Answer: I/O devices are thousands of times slower than the CPU. Spinning (busy waiting) would waste millions of CPU cycles doing nothing. By blocking the process (moving it to the **Waiting** state), the OS frees the CPU.

The OS performs a **Context Switch** to run a different, ready process. This keeps the CPU busy doing useful work for Process B while Process A waits for the disk, increasing overall system throughput.

Q4. Multiple processes try to print to the same printer simultaneously. Since the printer can only print one job at a time, where do the other requests go? Mention the "Kernel Data Structure" involved.

Answer: The device is busy. So, the OS Kernel maintains a **Device Status Table** and a corresponding **I/O Queue** for each device.

The incoming requests are encapsulated and placed into the **I/O Queue**. They wait their turn to be picked up by the scheduler/driver once the current job finishes.

Q5. Contrast Blocking I/O with Non-blocking I/O. Which approach would a high-performance web server (like Nginx) likely use to handle thousands of connections, and why?

Answer: In Blocking I/O, processes are suspended until I/O is done. Simple, but threads wait.

In Non-blocking I/O, the I/O call returns immediately, even if data isn't ready. The process continues execution.

For a Web Server, Non-blocking I/O is preferred. It allows a single process/thread to manage thousands of connections simultaneously. If it used blocking I/O, it would need thousands of threads (high overhead) to avoid freezing whenever one client had a slow network connection.

Lecture 9 - CPU Scheduling

Q1. Analyze the "CPU-I/O Burst Cycle" and its significance in the context of multiprogramming. How does the frequency distribution of CPU bursts influence the design of scheduling algorithms?

Answer: To understand why CPU scheduling is necessary, one must first understand the lifecycle of a process. Process execution is not a continuous block of calculation; rather, it consists of an alternating cycle of **CPU execution** (CPU burst) and **I/O wait** (I/O burst).

In a system without multiprogramming, when a process encounters an I/O burst (like reading from a file), the CPU sits idle, wasting valuable computational resources. The fundamental objective of multiprogramming is to maximize CPU utilization by switching the CPU to another process effectively during these wait times.

The **frequency distribution of CPU bursts** is critical for designing efficient schedulers. Empirical measurements show that processes tend to have a vast number of short CPU bursts (I/O-bound

programs) and a small number of long CPU bursts (CPU-bound programs). A scheduling algorithm must be designed to handle this curve effectively—balancing the needs of interactive processes (short bursts) against computational heavyweights (long bursts) to ensure the system remains responsive and efficient.

Q2. Differentiate between preemptive and non-preemptive scheduling by examining the specific process state transitions that trigger a scheduling decision. Why is non-preemptive scheduling considered "cooperative"?

Answer: CPU scheduling decisions occur at four specific transition points in a process's life:

1. When a process switches from **running to waiting** (e.g., an I/O request).
2. When a process switches from **running to ready** (e.g., an interrupt).
3. When a process switches from **waiting to ready** (e.g., I/O completion).
4. When a process **terminates**.

Non-preemptive scheduling occurs only under conditions 1 and 4. In this mode, once the CPU is allocated to a process, that process keeps the CPU until it voluntarily releases it by terminating or switching to a waiting state. It is "cooperative" because the scheduler waits for the process to yield.

Preemptive scheduling, on the other hand, can occur during conditions 2 and 3. Here, the OS can forcibly remove a running process from the CPU to allocate it to a higher-priority task. This approach is standard in modern operating systems (like Windows 95+ and Macintosh OS) to ensure responsiveness, whereas older systems like Windows 3.x relied on non-preemptive methods.

Q3. While preemptive scheduling allows for better responsiveness, it introduces the risk of "race conditions." Explain this phenomenon in the context of shared data and kernel mode operations.

Answer: Preemptive scheduling is powerful but dangerous because it interrupts a process arbitrarily. A **race condition** arises when multiple processes share data, and a process is preempted while it is in the middle of updating that data.

For example, if Process A is reading a shared variable and is preempted before it can finish writing a new value, Process B might run and read the data in an inconsistent or partially updated state.

This danger extends to the operating system kernel itself. If the kernel is modifying critical internal structures (like a process list) and is preempted by an interrupt, the kernel data could be corrupted. Consequently, a preemptive kernel requires sophisticated synchronization mechanisms, such as **mutex locks**, to ensure that shared kernel data structures are protected and that race conditions are prevented during these context switches.

Q4. The "Dispatcher" is a distinct component from the scheduler. Elucidate the specific responsibilities of the dispatcher and explain how "dispatch latency" impacts system performance.

Answer: It is important to distinguish between selection and execution. The **scheduler** selects which process *should* run next, but the **dispatcher** is the module that actually gives control of the CPU to that selected process.

The dispatcher performs the low-level mechanics of the hand-over, which involves:

1. **Switching context:** Saving the state of the old process and loading the state of the new one.
2. **Switching to user mode:** Transitioning from kernel mode back to user mode so the application can run.
3. **Jumping:** Moving the instruction pointer to the proper location in the user program to resume execution .

This process is not instantaneous. The time taken for the dispatcher to stop one process and start another is called **dispatch latency**. This latency is pure overhead—no useful work is done during this time. Therefore, minimizing dispatch latency is crucial; if it is too high, the system spends more time shuffling processes than actually executing them.

Q5. An Operating System designer must balance various scheduling criteria. Compare "Throughput" with "Turnaround Time," and discuss the specific optimization goals for a scheduler regarding Waiting Time and Response Time.

Answer: Different systems prioritize different metrics, and a perfect scheduler for one context might fail in another.

Throughput measures the volume of work done, specifically the number of processes that complete execution per time unit. It is a system-wide metric.

Turnaround Time, conversely, is a per-process metric. It measures the total time elapsed from the submission of a process to its completion.

While maximizing throughput and minimizing turnaround time are standard goals , interactive systems (like a desktop PC) often prioritize **Response Time**—the time from submission until the *first* response is produced. This ensures the user sees a reaction quickly, even if the task isn't fully finished.

Additionally, schedulers aim to minimize **Waiting Time**, which is the total time a process spends sitting in the ready queue. Since the scheduler cannot control how long a process executes (CPU burst) or waits for I/O, optimizing waiting time is often the most direct way a scheduler can improve efficiency.

Lecture 10 - CPU Scheduling Algorithms 1

Q1. The First-Come, First-Served (FCFS) algorithm is technically simple but can lead to the "Convoy Effect." Analyze this phenomenon and explain why FCFS is generally considered unsuitable for time-sharing systems.

Answer: FCFS is the simplest scheduling algorithm: processes are assigned the CPU in the exact order they request it. It is **non-preemptive**, meaning once a process gains the CPU, it retains control until it voluntarily terminates or waits for I/O.

The **Convoy Effect** occurs when a single CPU-bound process (a process with a very long CPU burst) effectively "clogs" the system. Several shorter I/O-bound processes may get stuck waiting behind this single long process. Instead of quickly executing their short bursts and returning to I/O, they idle in the ready queue, leading to poor resource utilization and high average waiting times .

This characteristic makes FCFS problematic for **time-sharing systems**. In such environments, it is critical that every user or process receives a share of the CPU at regular intervals to maintain responsiveness. FCFS cannot guarantee this, as one process can monopolize the CPU for an extended period, making the system appear unresponsive to other users .

Q2. Shortest-Job-First (SJF) is often cited as the "optimal" scheduling algorithm. Explain the metric by which it is considered optimal, and discuss the primary implementation challenge that prevents its direct use in general-purpose scheduling.

Answer: SJF is considered optimal because it yields the **minimum average waiting time** for a given set of processes. By aggressively prioritizing processes with the shortest next CPU burst, it clears the queue of small tasks quickly, which mathematically reduces the total time all processes spend waiting.

However, the algorithm faces a significant implementation hurdle: **knowing the future**. The operating system cannot know the exact length of the *next* CPU burst for a process before it executes. While the OS knows the process history, the future burst length is unknown. Therefore, SJF cannot be implemented exactly as defined; it must rely on statistical prediction methods to approximate the length of the next burst based on past behavior .

Q3. Since the exact length of the next CPU burst is unknown, schedulers use "Exponential Averaging" for prediction. Deconstruct the formula $\tau_{n+1} = \alpha \tau_n + (1 - \alpha) \tau_n$. What role does the parameter α play in this prediction model?

Answer:

To implement SJF, the scheduler predicts the next burst length (t_{n+1}) using an exponential average of the past. The formula combines two elements:

1. t_n : The *actual* length of the most recent CPU burst (recent history).
2. τ_n : The *predicted* length of the past interval (past history) .

The parameter α (where $0 \leq \alpha \leq 1$) acts as a weighting factor that controls the "memory" of the system.

- If $\alpha = 0$: The formula becomes $\tau_{n+1} = \tau_n$. The system ignores recent events entirely and assumes the next burst will be the same as the long-term average.
- If $\alpha = 1$: The formula becomes $\tau_{n+1} = t_n$. The system ignores all past history and assumes the next burst will be exactly the same as the very last one.

Commonly, α is set to **0.5**, effectively giving equal weight to the most recent observation and the historical average .

Q4. Distinguish between non-preemptive SJF and Preemptive SJF (SRTF). How does the scheduler handle a newly arriving process in the Shortest-Remaining-Time-First (SRTF) algorithm?

Answer: Standard SJF is non-preemptive; once a process starts, it runs until its burst completes, even if a shorter job arrives. **Shortest-Remaining-Time-First (SRTF)** is the preemptive variant of SJF.

In SRTF, the scheduler re-evaluates the queue whenever a new process arrives. It compares the burst time of the *newly arrived* process against the **remaining time** of the *currently running* process. If the new process has a shorter burst than what is left of the current process, the CPU is preempted (taken away) from the current process and given to the new one.

This approach can further reduce average waiting time compared to non-preemptive SJF because it allows short jobs to "cut in line" immediately rather than waiting for the current long job to finish.

Q5. Consider a set of processes P_1, P_2, P_3 arriving at the same time ($t=0$) with burst times of 24ms, 3ms, and 3ms respectively. Compare the Average Waiting Time (AWT) if they are scheduled using FCFS (order P_1, P_2, P_3) versus SRTF (order P_2, P_3, P_1). What does this comparison reveal about the sensitivity of FCFS?

Answer:

This scenario highlights the extreme sensitivity of FCFS to arrival order.

- **Case 1 (P_1, P_2, P_3):** P_1 (24ms) runs first. P_2 waits 24ms. P_3 waits 27ms (24+3).
 - $AWT = (0 + 24 + 27) / 3 = 17\text{ms}$.
- **Case 2 (P_1, P_2, P_3):** P_2 (3ms) runs first. P_3 waits 3ms. P_1 waits 6ms (3+3).
 - $AWT = (0 + 3 + 6) / 3 = 3\text{ms}$.

The comparison reveals that the AWT under FCFS is not minimal and can vary substantially depending on whether CPU-bound processes arrive before or after I/O-bound processes. A simple reordering of the same tasks reduced the waiting time by over 80%, illustrating why FCFS is risky for systems with variable burst lengths.

Lecture 11 - CPU Scheduling Algorithms 2

Q1. Priority Scheduling assigns the CPU based on rank rather than arrival time. Explain the convention used to define "highest priority" in most systems and discuss the critical problem of "Starvation" inherent to this algorithm. How does "Aging" resolve this issue?

Answer: In Priority Scheduling, a priority number (integer) is associated with each process. The convention typically followed is that a smaller integer denotes a higher priority; thus, the CPU is allocated to the process with the smallest integer value.

The inherent weakness of this algorithm is indefinite blocking, also known as starvation. In a heavily loaded system, a steady stream of higher-priority processes can prevent a low-priority process from ever gaining access to the CPU. A low-priority task could literally wait indefinitely without ever executing.

To solve this, schedulers implement a technique called Aging. This involves gradually increasing the priority of a process as it sits in the wait queue. For example, if a process's priority number is decremented (improved) every few minutes, it will eventually become the highest-priority process in the system and be executed, regardless of new arrivals.

Q2. Conceptually, Shortest-Job-First (SJF) can be categorized as a specific type of Priority Scheduling. Analyze the relationship between these two algorithms. What determines the "priority" in an SJF context?

Answer: SJF is essentially a specific application of the general Priority Scheduling mechanism. In standard priority scheduling, the priority is an arbitrary number assigned based on external criteria (like user importance or payment).

In the case of SJF, the priority is internally derived from the process's characteristics. Specifically, the priority is the inverse of the predicted next CPU burst time. The shorter the predicted burst, the higher the priority (or the lower the priority value) the process is assigned. Therefore, SJF creates an ordering where the "most important" jobs are simply the ones that can finish the quickest.

Q3. Round Robin (RR) scheduling is explicitly designed for time-sharing systems. Describe the mechanics of the RR algorithm, specifically focusing on the "Ready Queue" structure and the concept of the "Time Quantum."

Answer: Round Robin is effectively the First-Come, First-Served (FCFS) algorithm modified with preemption to ensure responsiveness in time-sharing environments .

The central mechanic of RR is the Time Quantum (or time slice)—a small unit of time, typically ranging from 10 to 100 milliseconds . The scheduler treats the ready queue as a circular queue. It iterates through this queue, allocating the CPU to each process for a time interval of up to one quantum.

If a process's CPU burst exceeds the time quantum, it is preempted (forced off the CPU) and placed at the back of the circular queue to wait for its next turn. This ensures that no single process can monopolize the processor, allowing the system to switch rapidly between processes .

Q4. The performance of the Round Robin algorithm is highly sensitive to the size of the Time Quantum. Contrast the system behavior when the quantum is "extremely large" versus when it is "extremely small." What is the impact on context switching?

Answer: The choice of time quantum dictates the nature of the scheduler:

- **Extremely Large Quantum:** If the quantum is large enough to cover the longest CPU burst in the system, the RR policy degenerates into a standard **FCFS policy**. The overhead of round-robin management remains, but the benefit of responsiveness is lost.
- **Extremely Small Quantum:** If the quantum is too small, the system suffers from excessive overhead. For example, if the quantum is 1 time unit and a process needs 10 units, the system must perform 9 context switches. Since a context switch takes time (typically < 10 microseconds), performing them too frequently wastes significant CPU time on administrative tasks rather than executing user code.

Thus, the quantum must be large enough relative to the context-switch time to be efficient, but small enough to maintain interactivity.

Q5. While "Turnaround Time" generally improves with shorter jobs, the relationship between Time Quantum size and Average Turnaround Time is not linear. What is the general rule of thumb regarding CPU bursts and Time Quantum sizing to optimize performance?

Answer: It might be intuitive to think that a larger time quantum always improves turnaround time by reducing overhead, but this is not strictly true. The average turnaround time can vary irregularly as the time quantum increases.

To optimize performance, the system aims to allow the majority of processes to finish their current CPU burst within a single time quantum. The general rule of thumb is that **80% of CPU bursts should be shorter than the time quantum**.

If the quantum covers 80% of bursts, most processes finish their task immediately without being preempted and waiting in the queue again. However, the quantum is still small enough to prevent the longest 20% of jobs from blocking the system, maintaining the benefits of time-sharing.

Lecture 13 - CPU Scheduling Algorithms 3

Q1. Multilevel Queue Scheduling introduces a rigid structure to the ready queue. Explain how this algorithm handles different classes of processes (e.g., interactive vs. batch) and discuss the two levels of scheduling decisions that must be made in this architecture.

Answer: Multilevel Queue Scheduling addresses the reality that different processes have distinct requirements. It partitions the ready queue into separate, permanent queues, typically distinguishing between **foreground processes** (interactive) and **background processes** (batch) .

In this architecture, the operating system must make scheduling decisions at two distinct levels:

1. **Intra-queue Scheduling:** Each specific queue can utilize a scheduling algorithm best suited for its process type. For example, the foreground queue often uses **Round Robin (RR)** to ensure responsiveness, while the background queue uses **First-Come, First-Served (FCFS)** for efficiency .
2. **Inter-queue Scheduling:** The scheduler must also decide which queue gets the CPU. This is often handled via **fixed priority scheduling**, where the foreground queue has absolute priority over the background queue (i.e., batch processes only run if the foreground queue is empty). However, to prevent the starvation of lower-priority queues, systems may employ **time slicing**, allocating a specific percentage of CPU time to each queue (e.g., 80% to foreground, 20% to background) .

Q2. How does the Multilevel Feedback Queue (MLFQ) scheduler improve upon the standard Multilevel Queue approach? specifically, detail the five parameters that define an MLFQ scheduler and explain how "aging" is implemented in this system.

Answer: The standard Multilevel Queue algorithm suffers from inflexibility because processes are permanently assigned to a queue. The **Multilevel Feedback Queue (MLFQ)** improves upon this by allowing processes to **move between queues** based on their behavior.

An MLFQ scheduler is defined by five key parameters:

1. The number of queues.

2. The scheduling algorithm used for each queue.
3. The method to determine when to **upgrade** a process to a higher-priority queue.
4. The method to determine when to **demote** a process to a lower-priority queue.
5. The method to determine which queue a process enters when it first needs service .

This flexibility allows the system to implement **aging** to prevent starvation. If a low-priority process waits too long, it can be moved to a higher-priority queue. Conversely, processes that consume too much CPU time (CPU-bound) can be demoted to lower-priority queues, leaving the high-priority queues for interactive (I/O-bound) processes.

Q3. Consider a Multilevel Feedback Queue system with three queues: Q_0 (RR, 8ms quantum), Q_1 (RR, 16ms quantum), and Q_2 (FCFS). Trace the path of a long CPU-bound process entering this system and explain how preemption is handled between these queues.

Answer: In this specific MLFQ configuration, a new process enters Q_0 and is served on a First-Come, First-Served basis. When it gains the CPU, it receives a time quantum of **8 milliseconds** .

If the process is CPU-bound and does not finish within this 8ms quantum, it is demoted to the tail of Q_1 . Once in Q_1 , it waits until Q_0 is empty. When it eventually runs, it receives a larger time quantum of **16 milliseconds**. If it still fails to complete its burst within this 16ms allocation, it is preempted again and demoted to Q_2 , where it is processed using a standard FCFS algorithm alongside other background tasks .

Preemption in this system is hierarchical. The scheduler prioritizes higher queues absolutely. If a process arrives for Q_1 , it will preempt a running process from Q_2 . Furthermore, a process running in Q_1 will itself be preempted if a new process arrives in Q_0 , ensuring the system remains responsive to new, short tasks .

Q4. In the context of Multiple-processor Scheduling, distinguish between Asymmetric Multiprocessing and Symmetric Multiprocessing (SMP). Why is SMP the standard for modern operating systems despite its higher complexity?

Answer:

Multiple-processor scheduling introduces complexity regarding how the OS manages tasks across available CPUs.

- **Asymmetric Multiprocessing:** This approach utilizes a "master-slave" relationship. A single processor (the master server) handles all scheduling decisions, I/O processing, and system activities, while the other processors execute only user code. This is conceptually simple because only one processor accesses system data structures, reducing the need for complex data sharing and synchronization .

- **Symmetric Multiprocessing (SMP):** In this approach, each processor is self-scheduling. Processors may share a common ready queue or manage their own private ready queues. This is significantly more complex because the OS must ensure that two processors don't choose the same process and that tasks aren't lost from the queue .

Despite the complexity, SMP is the standard for modern operating systems because it avoids the bottleneck of a single master server, allowing the system to scale efficiently with the hardware.

Q5. Explain the concept of "Processor Affinity" and its relationship to cache memory. How does this concept conflict with the goal of "Load Balancing" in an SMP system?

Answer: Processor Affinity refers to the strategy of keeping a process running on the same processor it previously used. This is driven by cache efficiency: when a process runs on a CPU, it populates that CPU's high-speed cache with its data. If the process migrates to a different CPU, the new CPU's cache must be repopulated, and the old cache data is invalidated, which is a high-cost operation .

There are two types of affinity:

- **Soft affinity:** The OS *attempts* to keep the process on the same CPU but doesn't guarantee it.
- **Hard affinity:** The OS allows a process to specify a subset of processors it effectively "must" run on.

This concept conflicts with **Load Balancing**, which aims to distribute workload evenly across all processors. Load balancing mechanisms, such as **Push migration** (moving tasks from overloaded to idle CPUs) and **Pull migration** (idle CPUs taking tasks from busy ones), inherently require moving processes between CPUs . Therefore, an OS must balance the benefits of cache retention (affinity) against the benefits of maximizing CPU utilization (load balancing).

Lecture 14 - CPU Scheduling Case Study

Q1. Trace the evolution of the Linux scheduler from the standard UNIX model to the Version 2.5 kernel. What were the primary deficiencies in the earlier models that the Version 2.5 O(1) scheduler aimed to solve, and what new problem did it inadvertently introduce?

Answer: The Linux scheduler has evolved significantly to handle modern hardware complexities. Initially, the Linux kernel utilized a variation of the standard UNIX scheduling algorithm. While functional, this early model had two major weaknesses: it lacked support for **SMP (Symmetric Multiprocessing)** systems and suffered from poor performance when handling a large number of processes .

To address these issues, the Version 2.5 kernel introduced an **O(1) scheduler**. This scheduler ran in constant time regardless of the number of tasks and provided robust support for SMP systems,

including features for processor affinity and load balancing. However, while it excelled at backend efficiency, it introduced a new problem: it resulted in **poor response times for interactive processes**, which are critical for desktop computer systems. This dissatisfaction led to the development of the Completely Fair Scheduler (CFS) in later versions.

Q2. The Completely Fair Scheduler (CFS), introduced in Linux 2.6.23, departs from traditional priority queues. Describe the data structure CFS uses to organize tasks and explain the role of the "vruntime" variable in selecting the next process to execute.

Answer: Unlike traditional schedulers that use fixed priority queues, the **Completely Fair Scheduler (CFS)** organizes all runnable tasks in a **balanced binary search tree** (specifically a Red-Black tree). The key used to sort this tree is a variable called **vruntime** (virtual run time).

vruntime records how long a task has run on the CPU, but it is adjusted based on the task's priority (nice value). The scheduler's logic is simple: it always picks the task with the **smallest vruntime**. In a balanced tree, this task is always found at the **leftmost node**. While inserting or deleting tasks takes $O(\log N)$ time, identifying the next task to run is very fast. When a task runs, its **vruntime** increases, eventually moving it to the right in the tree and allowing other tasks (with lower **vruntime**) to move to the left and take the CPU.

Q3. In the context of the CFS scheduler, explain how the "Nice Value" influences CPU allocation. How does the system ensure fairness between an I/O-bound task and a CPU-bound task that have the same nice value?

Answer: In CFS, the **Nice Value** (ranging from -20 to +19) determines the *proportion* of CPU time a task receives rather than a strict static priority. A lower numerical nice value indicates a higher relative priority. Tasks with lower nice values are assigned a higher decay rate for their virtual runtime, meaning their **vruntime** grows more slowly than that of lower-priority tasks, allowing them to stay at the "front" of the line (left side of the tree) longer.

When two tasks have the same nice value, CFS handles the distinction between I/O-bound and CPU-bound tasks naturally via **vruntime**.

- An **I/O-bound task** runs for short bursts and sleeps often, so its **vruntime** remains low.
- A **CPU-bound task** runs continuously, causing its **vruntime** to increase rapidly. Consequently, the I/O-bound task will almost always have a lower **vruntime** than the CPU-bound task. This gives the I/O task higher priority, allowing it to **preempt** the CPU-bound task immediately when it wakes up, ensuring the system remains responsive.

Q4. The Windows scheduling architecture relies on a "Dispatcher" and a tiered priority scheme. Detail the organization of the 32 priority levels in Windows and list the specific events that trigger the Dispatcher to perform a context switch.

Answer:

Windows uses a priority-based preemptive scheduling algorithm handled by a kernel component called the **Dispatcher**. It organizes threads into a **32-level priority scheme**:

- **Variable Class (1-15):** Used for standard user processes and applications.
- **Real-time Class (16-31):** Reserved for time-critical tasks.
- **Priority 0:** Reserved for the memory-management thread. The system maintains a queue for each priority level and runs the highest-priority thread available. If no user threads are runnable, it executes an idle thread .

The Dispatcher is triggered to select a new thread under specific conditions:

1. A thread is **preempted** by a higher-priority thread.
2. A thread **terminates**.
3. A thread's **time quantum ends**.
4. A thread calls a **blocking system call** (e.g., waiting for I/O).

Q5. Windows uses a dynamic priority adjustment mechanism for processes in the "Variable" class. Explain how "Priority Boosting" functions for foreground processes and for threads waiting on events. Why does the OS distinguish between foreground and background processes?

Answer: Windows attempts to optimize user experience by distinguishing between **foreground processes** (the window the user is currently interacting with) and **background processes**. To ensure responsiveness, Windows gives the foreground process a **3x priority boost** (specifically, it receives a time quantum three times longer than background processes).

Additionally, Windows applies dynamic boosts to threads in the variable class (levels 1-15) when they complete a wait operation. If a thread was waiting for I/O (like a keyboard press or disk read), its priority is boosted temporarily when that event occurs. This ensures that the thread can run immediately to process the data. After the boost, the priority decays back to the thread's **base priority**, ensuring it never drops below its assigned minimum.

Lecture 15 - Introduction to shell and cron

Q1. Analyze the operational flow of a Shell when a user enters a command. specifically, how does the shell distinguish between a "Shell Variable" and an "Environment Variable" in terms of scope and inheritance?

Answer: When a user types a command, the shell acts as an interface between the user and the kernel. It parses the input into tokens (command name, options, arguments) and locates the

executable—either in the current directory or by searching through the directories listed in the `$PATH` variable.

A critical distinction in shell operation is variable management:

- **Shell Variables:** These are local to the specific shell instance where they are defined. They are *not* passed down to child processes. They are often used for ephemeral data within a script or session .
- **Environment Variables:** These are defined for the current shell but are **inherited** by any child shells or processes spawned from it. This allows configuration data (like user preferences or system paths) to be passed seamlessly to applications .
- **Scope Transition:** A shell variable can be promoted to an environment variable using the `export` command (e.g., `export MY_VAR="value"`). Without `export`, a child shell (spawned by typing `bash`) would not see the variable.

Q2. Explain the significance of the `PATH` environment variable. What specific problem does it solve for the user, and how does the shell utilize it during command execution?

Answer: The `PATH` environment variable is a colon-separated list of directories that acts as a roadmap for the shell.

Without `PATH`, a user would have to type the absolute path of every program they wanted to run (e.g., `/usr/bin/ls` instead of just `ls`). The `PATH` variable solves this by automating the search. When a command is entered, the shell iterates through the directories listed in `PATH` in order. It executes the first matching executable it finds. This mechanism allows users to run system utilities and custom scripts from any location in the file system without knowing their exact installation path.

Q3. Deconstruct the syntax of `if` statements and `for` loops in a Bash script. How do the control flow keywords `break` and `continue` alter the execution of a loop?

Answer: Bash scripting relies on specific keywords to define blocks of code:

- **If Statements:** These begin with `if` and end with `fi`. The condition is typically placed in brackets `[ condition ]`, followed by a `then` block. An optional `else` or `elif` block can handle alternative cases .
- **For Loops:** These iterate over a list of items using the syntax `for variable in list`. The body of the loop is enclosed between `do` and `done` .

Control flow within loops is managed by:

- **`continue`:** This command skips the remaining commands in the *current* iteration of the loop and jumps immediately to the next iteration.

- **break:** This command terminates the loop entirely, exiting the **do-done** block and proceeding to the script instructions that follow the loop.

Q4. Define **cron and its relationship to the **crond** daemon. How does this architecture allow for unattended task execution compared to standard interactive programs?**

Answer:

Cron is a job scheduling utility in Unix-like systems designed to execute scripts or commands at specific times or intervals.

It relies on a background process called the **crond daemon**. A daemon is a program that runs continuously in the background, rather than under the direct control of an interactive user. The **crond** daemon starts when the system boots and wakes up every minute to check if any scheduled jobs (defined in "cron tables" or **crontab**) need to run .

This architecture allows for **unattended execution**: unlike interactive programs that require a user to be logged in and typing commands, **cron** jobs run automatically in the background, making them ideal for system maintenance, backups, or periodic logging.

Q5. You are tasked with scheduling a system backup script (backup.sh**) to run automatically. Analyze the fields of a crontab entry and construct the specific line required to execute this script every Wednesday at 4:30 AM.**

Answer: A **crontab** entry consists of five time-and-date fields followed by the command to execute. The fields, in order, are:

1. **Minute** (0-59)
2. **Hour** (0-23)
3. **Day of the Month** (1-31)
4. **Month of the Year** (1-12)
5. **Day of the Week** (0-6, where 0 is Sunday) .

To schedule **backup.sh** for **Wednesday at 4:30 AM**, the fields would be:

- **Minute:** 30
- **Hour:** 4 (24-hour format)
- **Day of Month:** * (every day of the month)
- **Month:** * (every month)
- **Day of Week:** 3 (Wednesday)

Final Crontab Entry:

30 4 * * 3 /path/to/backup.sh